

Layer 2: Launch Artifacts

Purpose: Represent an operationally faithful, scope-aware symbolic representation of ROS 2 launch semantics that can be extracted once, stored independently, and replayed deterministically for multiple configurations, with stable action identities enabling ground truth patching.

Overview

Layer 2 captures the canonical construction-time action Intermediate Representation which is, essentially, a symbolic launch program. One of its main goals is to be parsed once, cached, and replayed multiple times with different configurations.

Key Characteristics

- Launch actions represent potential instantiations, not actual nodes.
- Actions have hash-based IDs that are deterministic and stable across formatting and code movement changes.
- Inclusions of other launch files are not flattened; recursive resolution happens during replay.

Design Invariants

1. **Order Preservation:** Execution sequence is explicitly represented via ordered action ID lists.
 2. **Scope Preservation:** Nested scopes (namespace, parameters, arguments) are explicitly modeled through action hierarchy.
 3. **Symbolic Semantics Preservation:** No evaluation; conditions and values remain symbolic with abstract interpretation happening at higher Layers.
 4. **Ground Truth Compatibility:** Deterministic and stable action IDs can be easily referenced in ground truth annotations to support idempotent patching operations.
-

LaunchDescription

A LaunchDescription is the entry point for a launch file. It represents a complete launch file as a symbolic program with stable action identities. It contains a mapping of actions (for stable lookup) and a launch sequence (for execution order). This dual structure enables both efficient access and faithful replay while supporting ground truth patching.

Fields

Field	Type	Required	Description
<code>id</code>	string	Yes	Unique identifier
<code>launch_file_id</code>	string	Yes	File ID (Layer 0) of the launch file
<code>format</code>	enum	Yes	<code>python</code> , <code>xml</code> , <code>yaml</code>

Field	Type	Required	Description
<code>actions</code>	map[string, LaunchAction]	Yes	All actions keyed by action ID
<code>launch_sequence</code>	list[string]	Yes	Top-level action IDs in execution order
<code>provenance</code>	ElementProvenance	Yes	How discovered (includes source location)

Design Rationale

The `actions` map + `launch_sequence` list pattern provides:

- **Stable identity:** Actions keyed by deterministic ID (not position)
- **Explicit order:** `launch_sequence` preserves imperative semantics
- **Cheap reordering:** Moving actions doesn't change IDs
- **Ground truth support:** Can reference `action_id` reliably across extractions
- **Efficient lookup:** O(1) access to any action by ID

Example

```
{
  "id": "launch_desc_main",
  "launch_file_id": "file_main_launch_py",
  "format": "python",
  "actions": {
    "la:file_main:a3f5e2b4#0": {
      "id": "la:file_main:a3f5e2b4#0",
      "action_type": "declare_argument",
      "name": "use_camera",
      "default_value": {"type": "literal", "value": "true"},
      "provenance": {
        "extraction_method": "static_analysis",
        "source_location": {"file": "launch/main.launch.py", "line": 12},
        "confidence": 1.0
      }
    },
    "la:file_main:7c8a1e3f#0": {
      "id": "la:file_main:7c8a1e3f#0",
      "action_type": "node",
      "package": {"type": "literal", "value": "sensor_pkg"},
      "executable": {"type": "literal", "value": "camera_node"},
      "conditions": [{"eq", ["launch_arg_get", "use_camera"], "true"}],
      "provenance": {
        "extraction_method": "static_analysis",
        "source_location": {"file": "launch/main.launch.py", "line": 15},
        "confidence": 1.0
      }
    },
    "la:file_main:2d9e4f1a#0": {
```

```

    "id": "la:file_main:2d9e4f1a#0",
    "action_type": "include",
    "included_launch_id": "launch_desc_sensors",
    "argument_mappings": {
      "use_sensors": ["launch_arg_get", "use_camera"]
    },
    "provenance": {
      "extraction_method": "static_analysis",
      "source_location": {"file": "launch/main.launch.py", "line": 20},
      "confidence": 1.0
    }
  },
  "launch_sequence": [
    "la:file_main:a3f5e2b4#0",
    "la:file_main:7c8a1e3f#0",
    "la:file_main:2d9e4f1a#0"
  ],
  "provenance": {
    "extraction_method": "static_analysis",
    "source_location": {"file": "launch/main.launch.py", "line": 1},
    "confidence": 0.95
  }
}

```

Action ID Generation Specification

Action IDs must be deterministic and stable across code changes to support idempotent ground truth patching.

ID Format

```
la:<file_id>:<local_hash>#<ordinal>
```

Components:

- **la**: Prefix indicating launch action
- **<file_id>**: Layer 0 file ID (e.g., `file_main_launch_py`)
- **<local_hash>**: 8-character hex hash of action source snippet
- **<ordinal>**: 0-based counter for same hash (collision resolution)

Example IDs:

```

la:file_sensors:a3f5e2b4#0    # First action with hash a3f5e2b4
la:file_sensors:a3f5e2b4#1    # Second action with same hash (collision)
la:file_main:7c8a1e3f#0      # Different hash, different file

```

Hash Input Components

The `local_hash` is computed from the **normalized source snippet** of the action constructor call. In practice, this means computing a hash after removing whitespace, normalizing string quotes, sorting keyword arguments alphabetically and removing comments.

Hash Input Examples:

```
# Original source
Node(
    package='sensor_pkg',
    executable='camera_node',
    name='camera_driver'
)

# Normalized for hashing
Node(executable="camera_node",name="camera_driver",package="sensor_pkg")
```

```
# Original source
DeclareLaunchArgument(
    'use_camera',
    default_value='true',
    description='Enable camera'
)

# Normalized for hashing
DeclareLaunchArgument("use_camera",default_value="true",description="Enable camera")
```

Stability Guarantees

The hash is **stable to**:

-  Whitespace changes (formatting)
-  Comment changes
-  Keyword argument order changes
-  Quote style changes (single vs. double)
-  Code movement (line number not in hash)
-  Adding/removing other actions

The hash **changes** (intentionally) when:

-  Action type changes (`Node` → `GroupAction`)
-  Primary identifier changes (node name, argument name)
-  Substantive argument changes (package name, executable)

Collision Resolution

When multiple actions hash to the same value, use the ordinal suffix:

Collision Example:

```
# Loop creating multiple nodes (handled by for-in condition, not expansion)
for camera_id in [0, 1, 2]:
    Node(package='camera_pkg', executable='camera_node')
```

If this produces separate actions (e.g., via instrumented execution or abstract interpretation), IDs would be:

```
la:file_sensors:9d8c7b6a#0
la:file_sensors:9d8c7b6a#1
la:file_sensors:9d8c7b6a#2
```

However, with the unified condition IR approach, this is typically a **single action** with a loop condition.

Scope and Uniqueness

- **File-level uniqueness:** IDs unique within a launch file (file_id in prefix)
- **Global uniqueness:** Full ID unique across all launch files (includes file_id)
- **Deterministic:** Same source → same ID (assuming deterministic file_id and normalization)

LaunchAction (Abstract Base)

All launch elements are actions. LaunchAction is the abstract base type with concrete subtypes.

Fields

Field	Type	Required	Description
id	string	Yes	Deterministic hash-based action ID
action_type	enum	Yes	declare_argument, set_parameter, push_namespace, node, include, group
conditions	list[ConditionExpression]	No	Conditions for action execution
provenance	ElementProvenance	Yes	How discovered (includes source location)

Design Rationale

- Stable, hash-based IDs support ground truth patching.
- Conditions capture control flow; Boolean conditions filter contexts, while loop conditions expand to multiple evaluation contexts.
- Provenance distinguishes statically extracted actions from actions observed in instrumented execution or actions provided by ground truth annotations.

Action Types

- **declare_argument**: Declares launch argument
- **set_parameter**: Sets parameter in scope
- **push_namespace**: Introduces namespace scope
- **node**: Symbolic node instantiation point
- **include**: Includes another launch file
- **group**: Groups actions with optional scope modifications

DeclareArgumentAction

Declares a launch argument that can be referenced by other actions.

Fields (extends LaunchAction)

Field	Type	Required	Description
name	string	Yes	Argument name
default_value	LaunchSubstitution	No	Default value if not provided
description	string	No	Human-readable description
choices	list[any]	No	Valid argument values (enumeration)

Semantics

- Adds the argument to the current scope.
- Subsequent actions can reference this argument via **argument_reference** LaunchSubstitution.
- If a value is not provided at launch time, the default value should be used.
- If choices are specified, validation ensures that the actual value is in the valid choices.

Example

```
{
  "id": "la:file_sensors:a3f5e2b4#0",
  "action_type": "declare_argument",
  "name": "use_camera",
  "default_value": {"type": "literal", "value": "true"},
  "description": "Enable camera sensor",
  "choices": ["true", "false"],
  "provenance": {
    "extraction_method": "static_analysis",
    "source_location": {"file": "launch/sensors.launch.py", "line": 12},
    "confidence": 1.0
  }
}
```

SetParameterAction

Sets a parameter value in the current scope.

Fields (extends LaunchAction)

Field	Type	Required	Description
<code>name</code>	string	Yes	Parameter name
<code>value</code>	LaunchSubstitution	Yes	Parameter value (symbolic)
<code>target_scope</code>	enum	No	<code>local</code> (default), <code>global</code>

Semantics

- Mutates parameter context for current and nested scopes.
- Order matters: later `SetParameterAction` can override earlier ones.
- Parameters apply to `NodeActions` that execute after this action.

Target Scope

- `local`: Parameter applies to nodes instantiated in current scope and nested scopes.
- `global`: Parameter applies to all nodes (overrides global parameters in higher layers).

Example

```
{
  "id": "la:file_sim:7c8a1e3f#0",
  "action_type": "set_parameter",
  "name": "use_sim_time",
  "value": {"type": "literal", "value": true},
  "target_scope": "global",
  "provenance": {
    "extraction_method": "static_analysis",
    "source_location": {"file": "launch/simulation.launch.py", "line": 18},
    "confidence": 1.0
  }
}
```

PushNamespaceAction

Introduces a namespace scope for child actions.

Fields (extends LaunchAction)

Field	Type	Required	Description
<code>namespace</code>	LaunchSubstitution	Yes	Namespace to push (can be dynamic)

Field	Type	Required	Description
<code>children</code>	<code>list[string]</code>	Yes	Ordered child action IDs

Semantics

- Creates a nested execution context with the given namespace pushed onto the stack.
- Child actions execute in the new, nested context.
- After children complete, the namespace is popped (context is restored).

Example

```
{
  "id": "la:file_robot:2d9e4f1a#0",
  "action_type": "push_namespace",
  "namespace": {
    "type": "argument_reference",
    "argument_name": "robot_name"
  },
  "children": [
    "la:file_robot:1a2b3c4d#0",
    "la:file_robot:5e6f7g8h#0"
  ],
  "provenance": {
    "extraction_method": "static_analysis",
    "source_location": {"file": "launch/robot.launch.py", "line": 25},
    "confidence": 1.0
  }
}
```

NodeAction

A **NodeAction** is *not* a concrete node. It is a symbolic instruction to create a node with a set of properties, if the applicable conditions are satisfied. During resolution in higher layers (abstract interpretation or execution), **NodeActions** that pass condition evaluation produce concrete **Node** entities. The resolution step should use **package** and **executable** to lookup the corresponding **NodeImplementation** from Layer 1 (if available) or create a minimal stub implementation.

Fields (extends `LaunchAction`)

Field	Type	Required	Description
<code>package</code>	<code>LaunchSubstitution</code>	Yes	Package name (e.g., "camera_pkg")
<code>executable</code>	<code>LaunchSubstitution</code>	Yes	Executable name (e.g., "camera_node")
<code>name</code>	<code>LaunchSubstitution</code>	No	Node name (if not specified, uses executable name)

Field	Type	Required	Description
<code>namespace</code>	LaunchSubstitution	No	Node namespace (if not specified, uses context namespace)
<code>parameters</code>	map[string, LaunchSubstitution]	No	Node-specific parameter overrides
<code>remappings</code>	list[Remapping]	No	Topic/service/action name remappings
<code>ros_arguments</code>	list[string]	No	ROS-specific arguments (e.g., <code>--log-level</code>)
<code>launch_prefix</code>	string	No	Command prefix (e.g., <code>gdb</code> , <code>valgrind</code>)

Design Rationale

- Package/executable can be dynamic; Layer 1 lookup happens during instance resolution.
- All values can be dynamic expressions.
- If a `name` is specified, it overrides the default; otherwise, it uses the executable name as default.
- If `namespace` is specified, it is used; otherwise the namespace from the context stack applies.
- Any given `parameters` merge with (and override) context parameters and `ExecutionConfiguration` global parameters.

Remapping Structure

Field	Type	Description
<code>from</code>	string	Original topic/service/action name
<code>to</code>	LaunchSubstitution	Target name (can be dynamic expression)

Example

```
{
  "id": "la:file_sensors:9d8c7b6a#0",
  "action_type": "node",
  "package": {"type": "literal", "value": "camera_pkg"},
  "executable": {"type": "literal", "value": "camera_node"},
  "name": {"type": "literal", "value": "camera_driver"},
  "parameters": {
    "frame_rate": {"type": "literal", "value": 30}
  },
  "conditions": [
    ["eq", ["launch_arg_get", "use_camera"], "true"]
  ],
  "provenance": {
    "extraction_method": "static_analysis",
    "source_location": {"file": "launch/sensors.launch.py", "line": 35},
    "confidence": 1.0
  }
}
```

```
    }  
  }  
}
```

IncludeAction

The **IncludeAction** represents a nested launch context that is resolved recursively *only during runtime resolution*, at higher layers of the metamodel. This preserves the compositional structure of multi-file launch systems.

Fields (extends LaunchAction)

Field	Type	Required	Description
included_launch_id	string	Yes	LaunchDescription ID to include
argument_mappings	map[string, LaunchSubstitution]	No	Map included launch arguments to values

Design Rationale

- Includes are not flattened; this maintains launch file structure and promotes reusability.
- The parent launch description controls the included launch configuration.
- Nested execution contexts manifest during runtime resolution.

Example

```
{  
  "id": "la:file_main:2d9e4f1a#0",  
  "action_type": "include",  
  "included_launch_id": "launch_desc_sensors",  
  "argument_mappings": {  
    "use_camera": {  
      "type": "argument_reference",  
      "argument_name": "enable_vision"  
    },  
    "camera_fps": {  
      "type": "literal",  
      "value": 30  
    }  
  },  
  "conditions": [  
    ["feature_selected", "sensors"]  
  ],  
  "provenance": {  
    "extraction_method": "static_analysis",  
    "source_location": {"file": "launch/robot.launch.py", "line": 42},  
    "confidence": 1.0  
  }  
}
```

```
}  
}
```

GroupAction

A **GroupAction** is the fundamental scope boundary mechanism. It can optionally push namespace and/or set parameters that apply to all child actions.

Fields (extends LaunchAction)

Field	Type	Required	Description
children	list[string]	Yes	Ordered child action IDs
namespace	LaunchSubstitution	No	Namespace to push for children
set_parameters	map[string, LaunchSubstitution]	No	Parameters to set for children

Design Rationale

- It provides a single mechanism for namespace and parameter scoping.
- It can group actions without introducing scope changes (e.g., just for conditional execution).
- Groups can nest arbitrarily.

Example

```
{  
  "id": "la:file_sim:8b3c5e9f#0",  
  "action_type": "group",  
  "namespace": {"type": "literal", "value": "/simulation"},  
  "set_parameters": {  
    "use_sim_time": {"type": "literal", "value": true}  
  },  
  "children": [  
    "la:file_sim:1a2b3c4d#0",  
    "la:file_sim:5e6f7g8h#0"  
  ],  
  "conditions": [  
    ["eq", ["launch_arg_get", "simulation"], "true"]  
  ],  
  "provenance": {  
    "extraction_method": "static_analysis",  
    "source_location": {"file": "launch/main.launch.py", "line": 50},  
    "confidence": 1.0  
  }  
}
```

LaunchSubstitution (Discriminated Union)

Launch files support complex value expressions (substitutions, argument references, expressions). **LaunchSubstitution** abstracts these as a discriminated union that remains symbolic until runtime resolution. This is used throughout Layer 2 for flexible value specification. It loosely corresponds to ROS 2 launch system "substitutions" but generalized to any resolvable value.

Literal Value

```
{ "type": "literal", "value": <any> }
```

Example:

```
{"type": "literal", "value": 30}  
{"type": "literal", "value": "camera_driver"}  
{"type": "literal", "value": true}
```

Argument Reference

```
{  
  "type": "argument_reference",  
  "argument_name": string,  
  "default_value": <any>  
}
```

Example:

```
{  
  "type": "argument_reference",  
  "argument_name": "robot_namespace",  
  "default_value": "/robot"  
}
```

Environment Variable

```
{  
  "type": "environment_variable",  
  "variable_name": string,  
  "default_value": <any>  
}
```

Example:

```
{
  "type": "environment_variable",
  "variable_name": "ROS_DOMAIN_ID",
  "default_value": "0"
}
```

File Path

```
{
  "type": "file_path",
  "package": string,
  "relative_path": string
}
```

Example:

```
{
  "type": "file_path",
  "package": "sensor_pkg",
  "relative_path": "config/camera_params.yaml"
}
```

Expression (Substitution Tree)

```
{
  "type": "expression",
  "expression": <IRExpression>
}
```

Example:

```
{
  "type": "expression",
  "expression": [
    "if",
    ["eq", ["launch_arg_get", "simulation"], "true"],
    "/simulation/robot",
    "/real/robot"
  ]
}
```

Validation Rules

LaunchDescription Validation

1. **Actions map consistency:** All keys must match action `id` fields
2. **Sequence validity:** All IDs in `launch_sequence` must exist in `actions` map
3. **Reachability:** All actions must be reachable from `launch_sequence` (directly or as child of reachable action)
4. **No orphans:** No action should exist in map but be unreachable
5. **File reference:** `launch_file_id` must reference existing Layer 0 file

Action ID Validation

1. **Format compliance:** Must match `la:<file_id>:<hash>#<ordinal>` pattern
2. **Hash length:** `<hash>` must be exactly 8 hexadecimal characters
3. **Ordinal format:** `<ordinal>` must be non-negative integer
4. **File consistency:** `<file_id>` must match `launch_file_id` of containing LaunchDescription

Action Tree Validation

1. **Valid children:** All IDs in `children` lists must exist in `actions` map
2. **No cycles:** Action tree must be acyclic (no action can transitively contain itself)
3. **Scope actions only:** Only `push_namespace`, `group`, `include` can have `children` field

Condition Validation

1. **Well-formed IR:** All conditions must follow IR expression syntax
2. **Valid operators:** Only launch-compatible operators allowed
3. **Variable scope:** `var_get` references must be to variables in scope (set by outer `for` condition)
4. **Loop variables:** `for` loop variables must be unique within action's condition list

NodeAction Validation

1. **Package required:** `package` field must be present
2. **Executable required:** `executable` field must be present
3. **Valid substitutions:** All LaunchSubstitution fields must be well-formed

IncludeAction Validation

1. **Included launch exists:** `included_launch_id` must reference existing LaunchDescription
2. **No self-inclusion:** LaunchDescription cannot include itself (directly or transitively)
3. **Argument mapping validity:** Mapped argument names should match declared arguments in included launch

LaunchSubstitution Validation

1. **Type-specific validation:**
 - `literal`: Value must match expected type
 - `argument_reference`: Argument must be declared in scope
 - `file_path`: Package must exist

- **expression**: Expression must be well-formed IR
-

Design Rationale Summary

Why Map + Sequence Structure?

Benefits:

-  Stable identity (hash-based IDs independent of position)
-  Explicit order (**launch_sequence** preserves imperative semantics)
-  Cheap reordering (change sequence, IDs unchanged)
-  Ground truth compatible (stable references across changes)
-  Efficient lookup (O(1) by ID)

Why Hash-Based IDs?

Benefits:

-  Deterministic (same source → same ID)
-  Stable to formatting/movement
-  Unique (collision resolution with ordinal)
-  Traceable (links to source via hash)

Why Not Evaluate in Layer 2?

Layer 2 is **MIGHT** space; a symbolic program, not concrete execution.

If we evaluated:

-  Arguments → Need values from ExecutionConfiguration (Layer 3)
-  Conditions → Need feature selections (Layer 3)
-  Expressions → Need full context (runtime or abstract interpretation)

By staying symbolic:

-  Parse once, cache
-  Replay with different configurations
-  Model independent of execution choices
-  Abstract interpretation can analyze structure

Why Abstract Interpretation + Sandbox Execution Fallback?

Benefits:

-  Most cases are resolvable (static lists, simple conditionals, XML/YAML)
-  Handles complex cases (dynamic ranges, Python logic)
-  Layer 2 stays simple (doesn't need to expand everything)
-  Higher layers are accurate (actual execution when needed)

Strategic Value

Layer 2 provides value through:

1. **Traceability:** Hash-based IDs link actions to source ✓
2. **Variability Analysis:** Conditions and structure expose configuration space ✓
3. **Stable Patching:** Ground truth can reference actions reliably ✓
4. **Canonical IR:** Parse once, cache, resolve multiple times ✓
5. **Hybrid Resolution:** Abstract interpretation where possible, execution where needed ✓

With these properties, Layer 2 becomes:

A canonical construction-time action IR with traceable, stable identities and unified control flow representation.

This makes it genuinely valuable as an intermediate representation between source files and runtime architecture.